

GPU-Accelerated Deep Learning-Based ECG Signal Classification for Cardiac Abnormality Detection

Group 7 — Deep Learning Project

Team: Ronit Jana (Member 1), Ritam Jana (Member 2), Rishi Raj (Member 3), Rohan Adak (Member 4), Rishabh Jha (Member 5)

Abstract

This project classifies single ECG heartbeats as Normal or Abnormal using deep learning. We use the MIT-BIH Arrhythmia Database (PhysioNet), preprocess 44 usable records into 360-sample beat segments, and train two models on the same data: a 1D Convolutional Neural Network (1D-CNN) and a CNN-LSTM. The CNN-LSTM is also trained on CPU and on a GPU (Tesla T4 in Google Colab) to measure the speedup. On a record-wise test set of 15,897 beats from 7 unseen records, the 1D-CNN reached accuracy 0.8066 and abnormal-class F1 0.5166, while the CNN-LSTM reached accuracy 0.7605 and F1 0.4479. The GPU trained the CNN-LSTM about 8.76 times faster than the CPU over 5 epochs and ran inference about 7.43 times faster. A Streamlit demo connects the full pipeline. The output of the models is a classification result and not a clinical diagnosis.

1. Introduction

An electrocardiogram (ECG) records the electrical activity of the heart. Each heartbeat produces a characteristic waveform, and abnormal beats (for example ventricular or atrial premature beats) change the shape and timing of this waveform. Checking long ECG recordings by hand is slow, so automatic beat classification is a useful application of machine learning.

Deep learning is suitable here because ECG is a sequential signal and a convolutional network can learn waveform features directly from the samples, without hand-made features. A recurrent layer such as an LSTM can additionally learn how these features change over time. Deep networks also train much faster on a GPU, which is the second focus of this project.

The project demonstrates four connected ideas, as set in the project brief: ECG data handling, deep-learning-based classification, model evaluation, and GPU/CPU performance comparison.

The complete pipeline is: Dataset → Data Understanding → Preprocessing → Segmentation → Train/Validation/Test Split → 1D-CNN → CNN-LSTM → Evaluation → CPU vs GPU Comparison → Final Results → Demo.

2. Dataset (Member 1)

We use the MIT-BIH Arrhythmia Database from PhysioNet, downloaded with the WFDB Python package. Each record is about 30 minutes long, sampled at 360 Hz, and has two leads (MLII and V5 in record 100). Each record has three files: a .dat file (signal), a .hea file (header) and a .atr file (beat annotations placed at each R peak).

We use lead MLII (channel 0). Four paced records (102, 104, 107, 217) were excluded, so 44 of the 48 records are used.

Item	Value
Source	MIT-BIH Arrhythmia Database (PhysioNet)
Sampling rate	360 Hz
Records downloaded / used	48 / 44
Total beats	100,733
Normal beats	90,125
Abnormal beats	10,608 (10.5%)

Label definition agreed by the team:

- Normal (0): N, L, R, e, j
- Abnormal (1): A, a, J, S, V, E, F, /, f, Q

Non-beat annotation symbols (for example '+') are ignored. Note that bundle branch block beats (L and R) are placed in the Normal class by this definition.

The dataset is strongly imbalanced: normal beats are far more common than abnormal ones. This affects how the models must be trained and judged (Section 6).

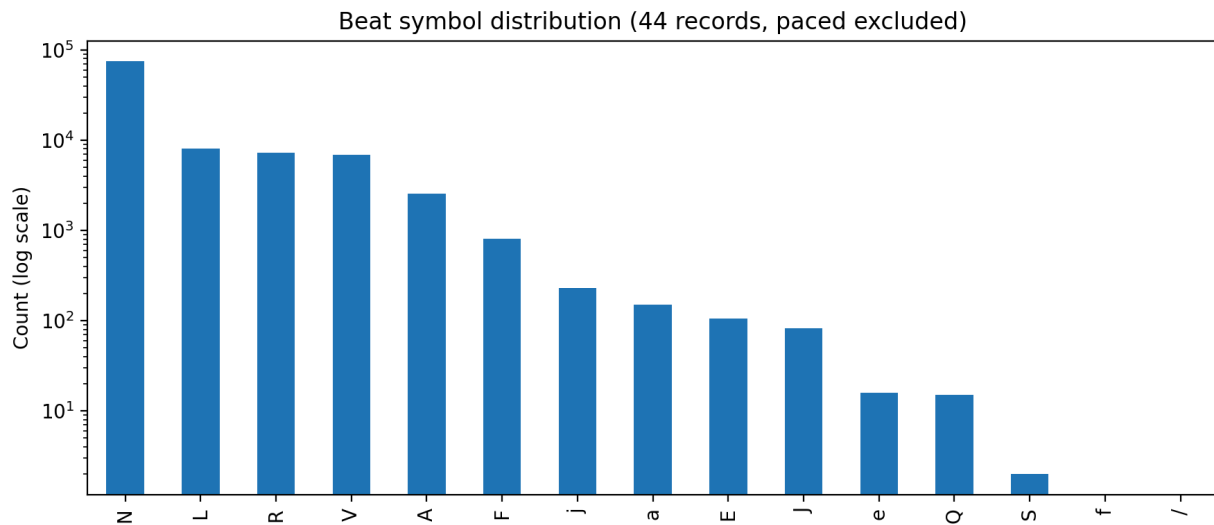


Figure 1: class_distribution.png — beat symbol counts, log scale

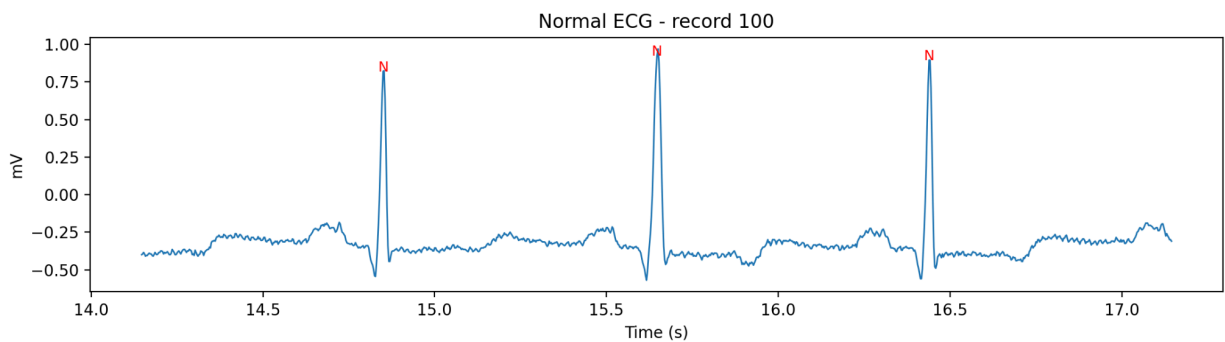
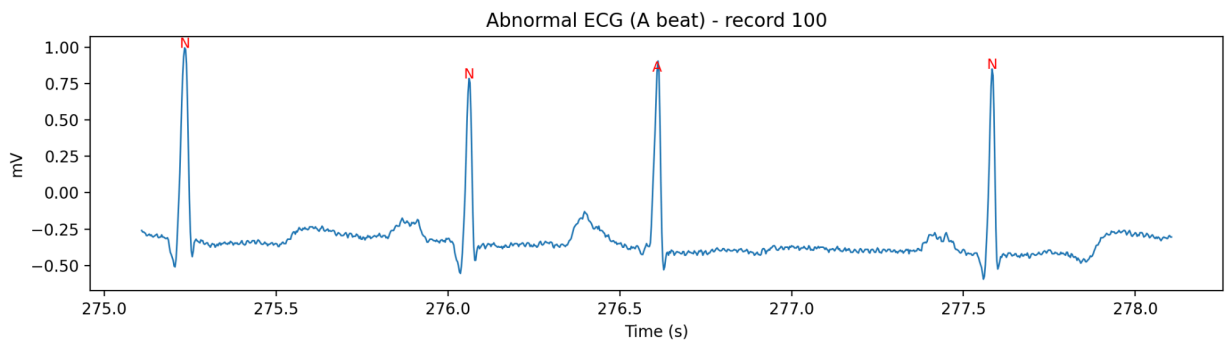


Figure 2: ecg_normal.png — normal ECG example (record 100)



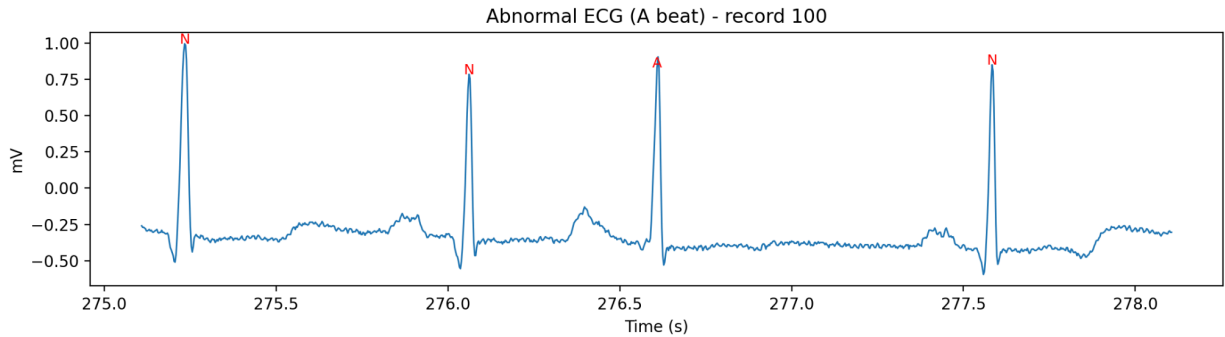


Figure 3: ecg_abnormal_V.png and ecg_abnormal_A.png — ventricular beat (record 119) and atrial beat (record 100)

3. Preprocessing and Data Split (Member 2)

The raw records were converted into fixed-size model inputs in four steps.

1. Filtering: a 4th-order Butterworth bandpass filter from 0.5 Hz to 45 Hz removes baseline wander and high-frequency noise. It is applied to the whole record first.
2. Segmentation: a 360-sample window (1 second) is cut around each annotated R peak, 180 samples before and 180 after. Beats whose window does not fit inside the record are dropped, so 100,733 annotated beats become 100,682 segments.
3. Normalization: each segment is z-score normalized, $(\text{segment} - \text{mean}) / \text{std}$, so the model receives consistently scaled input.
4. Labelling: each beat gets a binary label (Normal 0 / Abnormal 1) from its annotation symbol.

The split is record-wise (patient-wise): a record appears in only one of train, validation or test. This avoids data leakage, because beats from the same person look very similar and would otherwise inflate test results.

Split	Record s	Beats	Normal	Abnormal	Abnormal %
Train	30	68,386	62,790	5,596	8.2%
Validation	7	16,399	14,553	1,846	11.3%

Test	7	15,897	12,733	3,164	19.9%
------	---	--------	--------	-------	-------

The processed data was saved as `ecg_processed_v1.npz` (about 134 MB) with arrays `X_train` (68386, 360, 1), `X_val` (16399, 360, 1) and `X_test` (15897, 360, 1) in float32, with matching int32 label arrays. Members 3 and 4 both used this same file.

The abnormal share is 8.2% in train but 19.9% in test. This difference between splits is important when reading the results.

4. 1D-CNN Model (Member 3)

A 1D-CNN is suitable for ECG because a Conv1D layer slides small filters along the signal and learns local waveform patterns (such as the shape of a QRS complex), and pooling reduces the length while keeping the strongest responses.

Architecture: $\text{ECG} (360 \times 1) \rightarrow \text{Conv1D}(32, \text{kernel } 5) + \text{MaxPool} \rightarrow \text{Conv1D}(64, \text{kernel } 5) + \text{MaxPool} \rightarrow \text{Flatten} \rightarrow \text{Dense}(64) \rightarrow \text{Dropout}(0.5) \rightarrow \text{Sigmoid}$. The model has 379,265 parameters. The single sigmoid output is the probability that the beat is abnormal; a threshold of 0.5 gives the class.

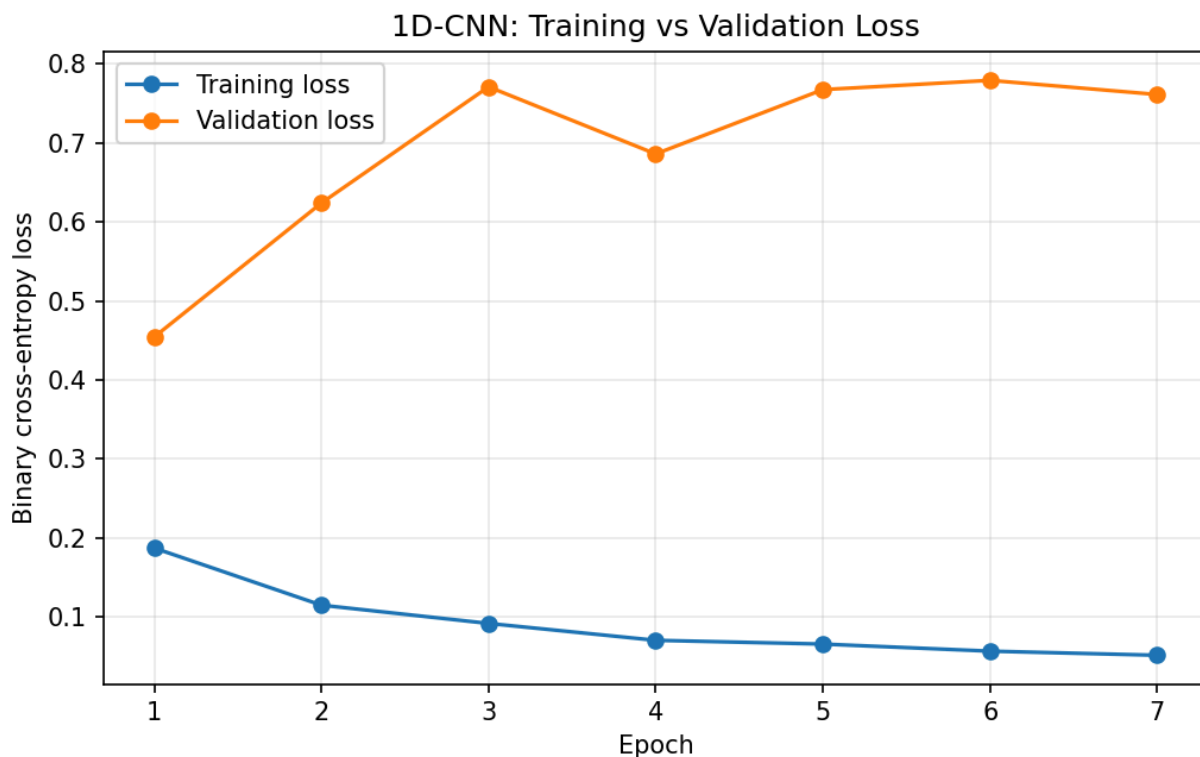
Setting	Value
Seed / batch size	42 / 128
Optimizer	Adam, learning rate 0.001, with ReduceLROnPlateau (factor 0.5, patience 2, min 1e-5)
Loss	Binary cross-entropy
Class weights	Normal 0.5446, Abnormal 6.1103 (balanced, from the train set)

Epochs Maximum 30, early stopping on val_loss (patience 6, best weights restored)

Threshold 0.5

Training used only the train and validation sets. The test set was used once, for the final evaluation of the best checkpoint. Early stopping stopped after 7 epochs and restored the weights of epoch 1, which had the best validation loss (0.4543, validation accuracy 0.8145). After epoch 1, training accuracy rose to about 98.8% while validation loss increased, which shows overfitting to the training records. The 1D-CNN run had no GPU visible and took 462.6 seconds.

Test results: accuracy 0.8066, precision (abnormal) 0.5139, recall (abnormal) 0.5193, F1 (abnormal) 0.5166, macro F1 0.6978, specificity 0.8780, ROC-AUC 0.8448, test loss 0.5674. Confusion matrix: TN 11179, FP 1554, FN 1521, TP 1643.



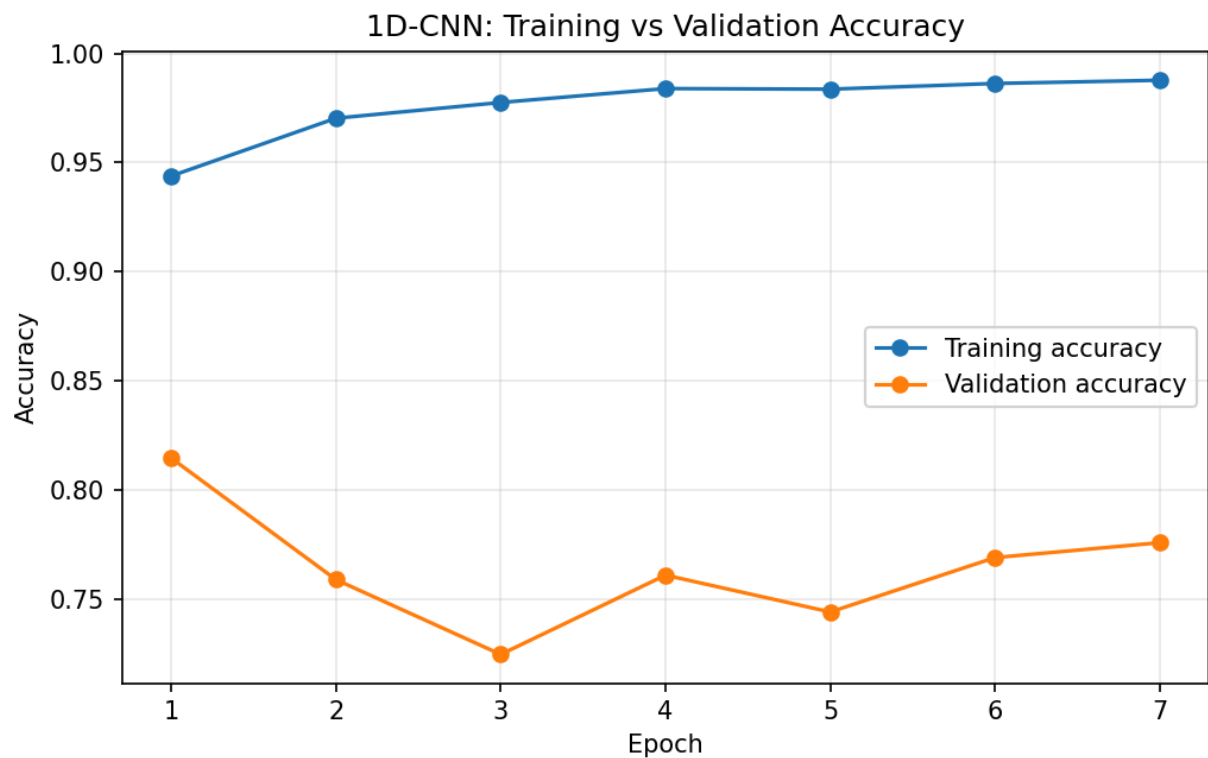


Figure 4: 1d_cnn_accuracy.png and 1d_cnn_loss.png — training vs validation

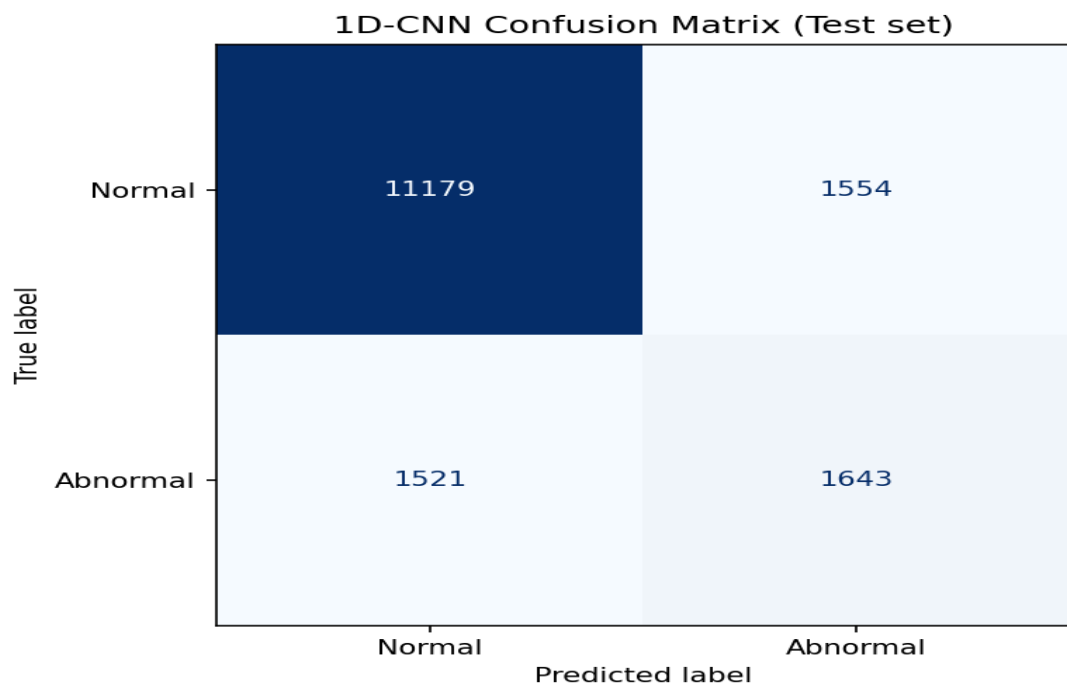


Figure 5: 1d_cnn_confusion_matrix.png — 1D-CNN confusion matrix on the test set

5. CNN-LSTM Model and CPU vs GPU Experiments (Member 4)

5.1 Architecture

The CNN-LSTM first uses convolution layers to extract local waveform features, and then an LSTM layer to learn how these features change along the beat. Three pooling steps reduce the 360 samples to 45 time steps, which are fed to the LSTM.

Architecture: ECG (360×1) $\rightarrow 3 \times$ (Conv1D + MaxPool) with 32, 64 and 128 filters $\rightarrow$ LSTM(64) $\rightarrow$ Dense(32) $\rightarrow$ Dropout(0.3) $\rightarrow$ Sigmoid (Normal / Abnormal).

5.2 Experiment settings

Setting	Value
Data	ecg_processed_v1.npz (same file as the 1D-CNN)
Seed / batch size	42 / 128
Optimizer	Adam, learning rate 0.001
Loss	Binary cross-entropy
Class weights	Normal 0.5446, Abnormal 6.1103
Epochs	Maximum 15, early stopping on val_loss (patience 4, best weights restored)
Threshold	0.5

Precision float32 (no mixed precision)

5.3 Main run (Experiment A, trained on GPU)

The model trained for 13 epochs with the best epoch at 9, taking 112.11 seconds (about 8.59 seconds per epoch). Test results: accuracy 0.7605, precision (abnormal) 0.4138, recall (abnormal) 0.4880, F1 (abnormal) 0.4479, specificity 0.8282, ROC-AUC 0.7418. Confusion matrix: TN 10546, FP 2187, FN 1620, TP 1544.

On the validation set, recall was 0.8483, but on the test set it was 0.4880. Training accuracy reached about 98% and the validation loss was unstable, so the model fits the training records much better than new, unseen records.

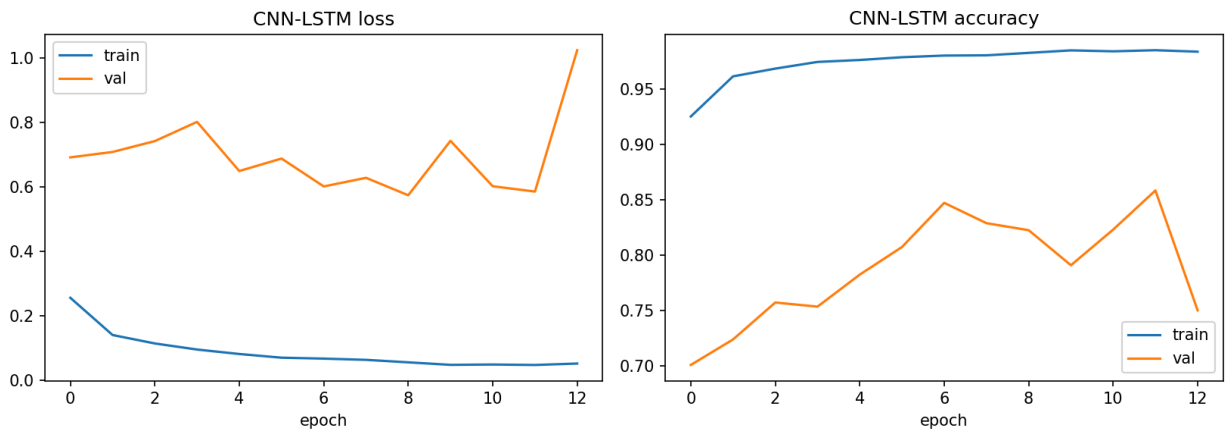
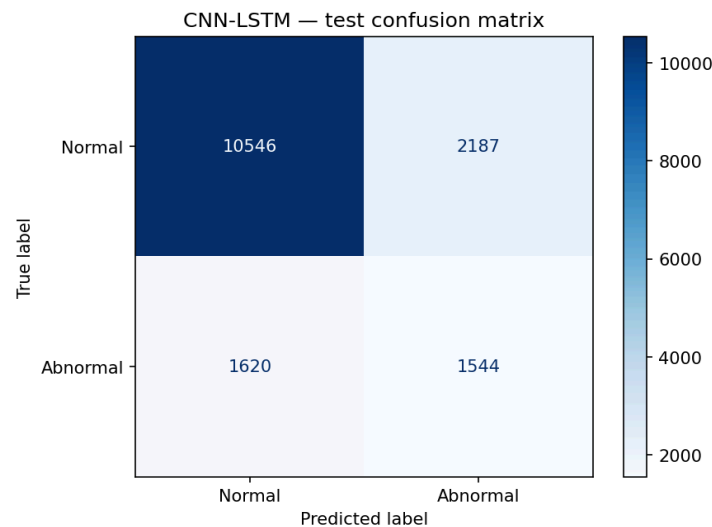


Figure 6: cnn_lstm_training_curves_v1.png — CNN-LSTM training and validation loss/accuracy



cnn_lstm_confusion_matrix_v1.png — CNN-LSTM confusion matrix on the test set

5.4 CPU vs GPU (Experiment B)

To measure the effect of the GPU, the same model, data, seed, batch size and number of epochs (5, no early stopping) were run once on the CPU and once on the GPU. The CPU run uses `tf.device('/CPU:0')` inside the same Colab runtime. Epoch time includes validation time.

Runtime used (recorded 2026-10-04): Google Colab, Tesla T4 GPU (15360 MiB), NVIDIA driver 580.82.07, CUDA 12.5.1, cuDNN 9, Intel Xeon CPU @ 2.00GHz (2 logical cores), about 12.7 GiB RAM, Python 3.13.15, TensorFlow 2.20.0, Keras 3.13.2.

Metric	CPU	GPU	Speedup (CPU / GPU)
Total training time (5 epochs)	297.20 s	33.94 s	8.76x
Average epoch time (epoch 2 onward)	60.64 s	6.22 s	9.75x
First epoch (includes warm-up)	54.47 s	8.80 s	-
Inference, full test set (15,897 beats)	3.563 s	0.479 s	7.43x
Inference per beat	0.2241 ms	0.0302 ms	7.42x
Test accuracy after 5 epochs	0.7632	0.7671	-

A GPU has thousands of small cores that perform many multiplications in parallel, which suits the matrix and convolution operations of neural networks, so the GPU is faster in every measurement. The CPU and GPU accuracy differ slightly (0.7632 and 0.7671) because they are separate 5-epoch runs with different compute kernels; these values also differ from the main run (0.7605), which used early stopping.

Limitation: each timing experiment was run once (TIMING_REPEATS = 1), and Colab hardware can differ between sessions, so timings should only be compared with runs on the same hardware.

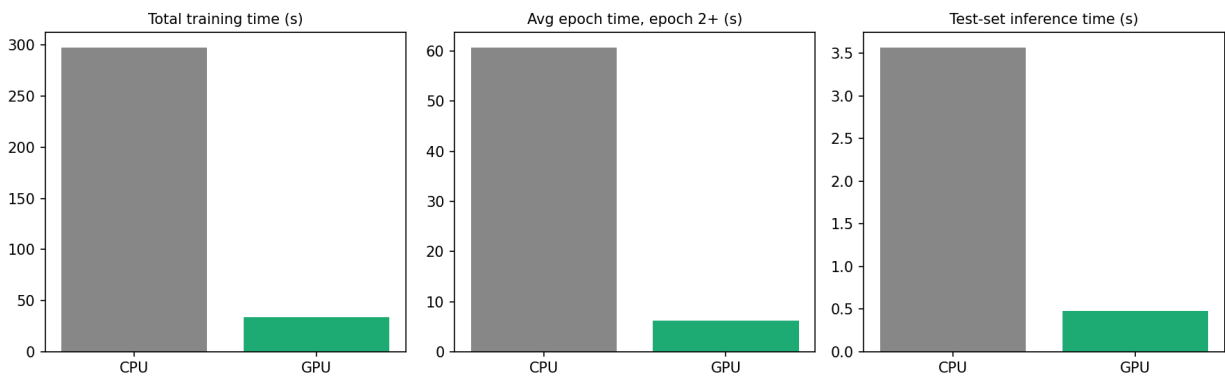


Figure 8: cnn_lstm_cpu_vs_gpu_v1.png — CPU vs GPU training, epoch and inference time

6. Model Comparison and Discussion (Member 5)

Both models were evaluated on the same test set (15,897 beats from 7 unseen records) with threshold 0.5.

Metric	1D-CNN	CNN-LSTM
Accuracy	0.8066	0.7605
Precision (abnormal)	0.5139	0.4138
Recall (abnormal)	0.5193	0.4880
F1-score (abnormal)	0.5166	0.4479
Specificity	0.8780	0.8282

ROC-AUC	0.8448	0.7418
Confusion matrix (TN / FP / FN / TP)	11179 / 1554 / 1521 / 1643	10546 / 2187 / 1620 / 1544
Training time	462.6 s (7 epochs, no GPU visible)	112.11 s (13 epochs, Tesla T4 GPU)

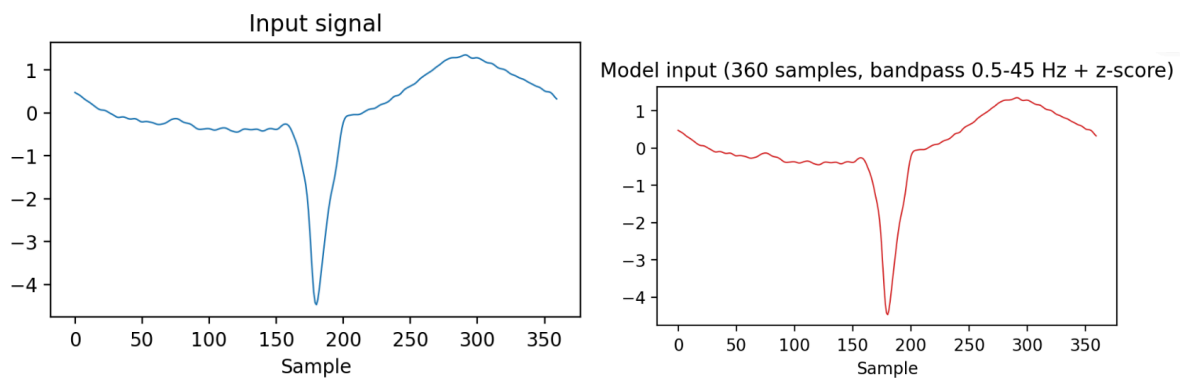


Figure 9: final_model_comparison.png — 1D-CNN vs CNN-LSTM test metrics and CNN-LSTM CPU vs GPU timing

Discussion, based only on the measured values:

- The 1D-CNN is better than the CNN-LSTM on every metric in the table. The extra LSTM layer did not improve the result on this dataset.
- Both models are weak on the abnormal class (F1 about 0.45 to 0.52). Accuracy alone is misleading here because 80.1% of the test beats are normal, which is why precision, recall, F1 and the confusion matrix are reported.
- Both models overfit the training records, and validation performance does not carry over to the test records. The test set has 19.9% abnormal beats compared with 8.2% in train, and the test records belong to unseen patients.
- The test result depends strongly on a few records: records 232, 233 and 223 contain 2,791 of the 3,164 abnormal test beats.
- The training times of the two models must not be compared directly. The 1D-CNN run had no GPU and different early-stopping settings, while the CNN-LSTM main run used the T4 GPU.
- The two models were trained with the same data, seed, batch size, optimizer, loss, class weights and threshold, but not identical epoch limits (30 vs 15), early-stopping patience

(6 vs 4) and learning-rate schedule (ReduceLROnPlateau only for the 1D-CNN). Each model was run once, so small differences should not be over-interpreted.

- The GPU gives about 7.4 to 9.8 times speedup for the CNN-LSTM, depending on whether total training, steady epoch time or inference is measured.

7. Demo (Member 5)

A Streamlit application (demo/app.py) connects the whole pipeline: input ECG beat → preprocessing → 1D-CNN and CNN-LSTM → predicted class (Normal or Abnormal).

1. Input: the user selects a beat from the test set of ecg_processed_v1.npz (the true label is shown), or uploads their own ECG file (.csv, .txt or .npy).
2. Preprocessing: bandpass filter 0.5 to 45 Hz, 360-sample window and per-segment z-score, the same as Section 3.
3. Prediction: both models output P(abnormal); a value of 0.5 or more means Abnormal. The app also shows whether the two models agree.
4. Evaluation: metrics, confusion matrices and the CPU vs GPU table are read directly from the results files, so no number is typed into the code.

The models were trained on MIT-BIH beats (MLII lead, 360 Hz), so other leads, sampling rates or filters give unreliable predictions. The demo is an educational project demo, not a medical device.

8. Conclusion and Future Work

We built one connected pipeline for ECG beat classification: dataset analysis, preprocessing with a record-wise split, a 1D-CNN, a CNN-LSTM, evaluation, a CPU vs GPU comparison, and a working demo.

On unseen records, the 1D-CNN performed better than the CNN-LSTM (F1 0.5166 vs 0.4479, ROC-AUC 0.8448 vs 0.7418), and neither model detects abnormal beats reliably yet. The GPU (Tesla T4) reduced CNN-LSTM training time from 297.20 s to 33.94 s (8.76x) and full test-set inference from 3.563 s to 0.479 s (7.43x).

Possible future work: re-run both models with identical epoch and early-stopping settings and on the same GPU runtime; repeat the timing experiments several times and report the average; use cross-validation across records so that results do not depend on a few test records; use stronger regularization or data augmentation to reduce overfitting; and try a finer class definition than the binary Normal / Abnormal label. Model output is a classification result, not a clinical diagnosis.

References

1. Moody GB, Mark RG. The impact of the MIT-BIH Arrhythmia Database. IEEE Engineering in Medicine and Biology Magazine, 20(3):45–50, 2001.
2. Goldberger AL, et al. PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. Circulation, 101(23):e215–e220, 2000.
3. WFDB Python package, PhysioNet. <https://github.com/MIT-LCP/wfdb-python>
4. TensorFlow and Keras documentation. <https://www.tensorflow.org> and <https://keras.io>
5. Group 7 project repository: README.md, results/.csv and results/.json (all numbers in this report are taken from these files).